

Embedded-Focused Study Plan with FSM and Automata Integration

stevemac321

Overview

This study plan focuses on embedded systems development using **C**, **C++**, and **Rust**. It includes foundational data structures and algorithms from **CLRS** and **Foundations of Computer Science** (FCS) that are applicable to embedded systems, including a deep dive into **Finite State Machines (FSMs)**, **automata theory**, and practical microcontroller tasks. The plan is prioritized to ensure a well-rounded development of both theoretical and practical skills for embedded system design.

Priority 1: Core Data Structures and Algorithms for Embedded Systems

1.1 Array-Based Data Structures

- Revisit **Heap Sort**:
 - Implement in C++ and ensure a static array-based structure.
 - Rust is suitable here for implementing priority management without dynamic memory.
- Implement **Priority Queues**:
 - Array-based implementation to prioritize real-time events.
 - **Rust is highly suitable** due to its memory safety features and efficient handling of static arrays.
- Explore **Circular Buffers** (Ring Buffers) for real-time data storage:
 - Use in scenarios like sensor data collection or task queuing.
 - Study sliding window algorithms for efficient real-time processing.
 - **Rust is ideal** here for implementing efficient circular buffers with static memory guarantees.

1.2 Graph Algorithms (with Hardware Focus)

- Implement **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** in both C++ and Rust:
 - Integrate with real-world scenarios such as hardware interrupt handling.
 - Use static adjacency matrices or lists for embedded performance.
 - **Rust is ideal** for these graph algorithms due to its safety and handling of recursion (especially DFS).
- Implement **Dijkstra's Algorithm** for shortest path finding:
 - Rust is suitable, offering safe memory management for complex algorithms like Dijkstra's.
- Study and implement **Kruskal's and Prim's Algorithms** for spanning tree problems.

1.3 Finite State Machines (FSMs) and Automata

- **Study and Implement FSMs** in embedded systems:
 - Revisit *state machines* and *automata theory* from *Foundations of Computer Science* (Chapter 10.2-10.4).
 - Focus on *deterministic* and *nondeterministic automata* (Chapter 10.3 and 10.4) to gain deeper understanding.
 - Implement both types of FSMs in *C* and *Rust* for event-driven systems like *device control* and *protocol handling*.
- Implement FSMs for real-time embedded applications:
 - Design FSMs in *C* and *Rust* to handle:
 - * *Device Control* (e.g., power states, operational modes).
 - * *Protocol Handling* (communication protocols with state-based input processing).
 - * *Interrupt Management* (state transitions based on hardware interrupts).
- **From Regular Expressions to Automata and Vice Versa:**
 - Study Chapter 10.8 and 10.9 from *Foundations of Computer Science* to understand how regular expressions map to automata.
 - Implement regular expression matching in Rust and relate it to automata theory.
- Explore *hierarchical state machines (HSMs)* for complex systems:
 - Study more advanced designs where nested states improve control flow for embedded tasks with multiple operational modes.

Priority 2: Microcontroller Hardware Tasks

2.1 Temperature, Voltage Monitoring, and Watchdog Timers

- **Temperature and Voltage Monitoring:**
 - Use internal sensors, and store data in a circular buffer.
 - Implement priority queues to track highest and lowest readings.
 - These tasks are hardware-specific and best implemented in C for microcontroller access.
- **Watchdog Timers:**
 - Implement timers in C, log reset events in a circular queue, and track system health.

2.2 Real-Time Clock (RTC) and Timestamp Handling

- Implement **binary search trees** for querying events based on timestamps.
- Develop **scheduling algorithms** (round-robin, priority-based) to handle tasks based on time events.
- Suitable for C++, though Rust can also handle timestamp management.

2.3 Signal Processing and Error Detection

- Study **Fast Fourier Transform (FFT)** for real-time signal processing.
- Implement **Cyclic Redundancy Check (CRC)** and **Hamming Codes** for fault detection and correction.
- Both C and Rust are good options, with Rust offering safety in buffer management.

Priority 3: Advanced Algorithms and Data Structures

3.1 Dynamic Programming with Embedded Constraints

- Implement basic **dynamic programming algorithms** (Knapsack, Fibonacci) using array-based approaches.
- Explore space-efficient implementations suitable for embedded systems.
- Rust is highly suitable for ensuring memory safety in dynamic programming.

3.2 Real-Time Scheduling and Task Management

- Study and implement **real-time scheduling algorithms**:
 - Focus on round-robin, earliest deadline first (EDF), and other algorithms.
- Integrate task management with hardware interrupts and timers.
- Best suited for C/C++ for low-level control, though Rust can handle scheduling in embedded OS environments.

Priority 4: Embedded-Specific Data Structures

4.1 Array-Based Implementations

- **Tries**:
 - Implement as array-based structures for string pattern matching.
 - **Rust** is highly suitable for implementing tries due to its strong handling of recursive data structures.
- **B-Trees**:
 - Study array-based or fixed-memory B-tree implementations for embedded databases.
 - Best suited for C++, though Rust can be used for its safety in managing complex tree structures.

Conclusion

This plan integrates both the core algorithms and data structures from **CLRS** and **Foundations of Computer Science**, emphasizing **FSMs**, **automata**, and graph algorithms, alongside hardware-specific tasks. FSM design is central to embedded systems, and this plan includes practical implementations in both **C** and **Rust** for real-time system management. Topics such as **regular expressions**, **automata**, and **state machines** are tied to both theoretical foundations and practical applications in protocol handling, control systems, and task scheduling.